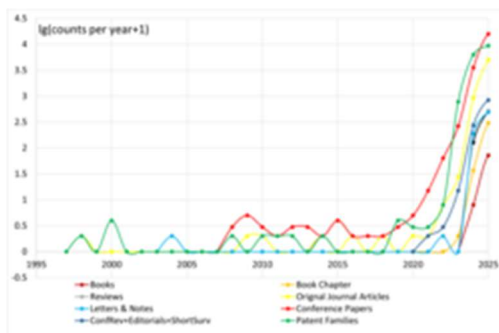


A **large language model (LLM)** is a [neural network](#) trained on a vast amount of text for [natural language processing](#) tasks, especially [language generation](#). LLMs can typically generate, summarize, translate, and analyze text in many contexts, and are a foundational technology behind modern [chatbots](#).^[1] Biased or inaccurate training data can make an LLM's output less reliable.^[2]

LLMs are typically based on [transformer](#) architecture.^[3] [Generative pre-trained transformers](#) (GPTs) are a type of LLM that is pre-trained to predict the next word.^[4] GPTs are then often [fine-tuned](#) to follow instructions and to behave as assistants.^[5]

[Benchmark](#) evaluations for LLMs attempt to measure [model reasoning](#), factual accuracy, [alignment](#), and [safety](#).^[6]

History



The number of publications about large language models



by year grouped by publication types

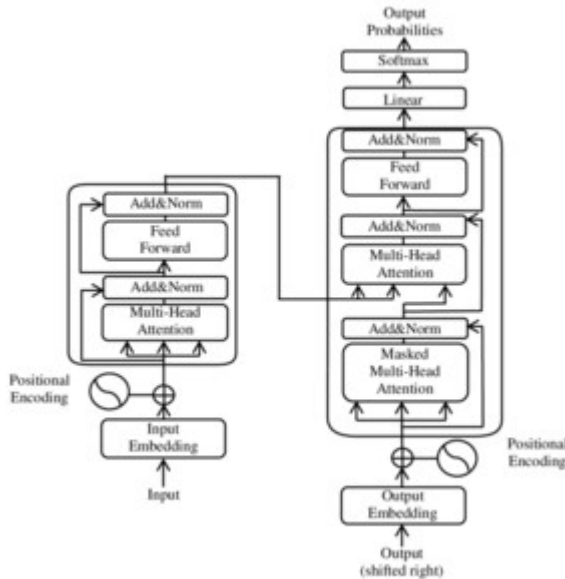
The training compute of notable large models in FLOPs vs publication date over the period 2010–2024. For overall notable models (top left), frontier models (top right), top language models (bottom left) and top models within leading companies (bottom right). The majority of these models are



language models. The training compute of notable large AI models in FLOPs vs publication date over the period 2017–2024. The majority of large models are language models or multimodal models with language capacity.

Before the emergence of transformer-based models in 2017, some [language models](#) were considered large relative to the computational and data constraints of their time. In the early 1990s, [IBM's statistical models](#) pioneered [word alignment](#) techniques for machine translation, laying the groundwork for [corpus-based language modeling](#). In 2001, a smoothed [n-gram model](#), such as those employing [Kneser–Ney smoothing](#), trained on 300 million words, achieved state-of-the-art [perplexity](#) on benchmark tests.^[7] During the 2000s, with the rise of widespread [internet access](#), researchers began compiling massive text datasets from the web ("web as corpus"^[8]) to train statistical language models.^{[9][10]}

Moving beyond *n*-gram models, researchers started in 2000 to use neural networks as language models.^[11] Following the breakthrough of [deep neural networks](#) in [image classification](#) around 2012,^[12] similar architectures were adapted for language tasks. This shift was marked by the development of [word embeddings](#) (e.g., [Word2Vec](#) by [Mikolov](#) in 2013) and sequence-to-sequence ([seq2seq](#)) models using [LSTM](#). In 2016, Google transitioned its translation service to [neural machine translation](#) (NMT), replacing statistical phrase-based models with deep [recurrent neural networks](#). These early NMT systems used LSTM-based [encoder-decoder architectures](#), as they preceded the invention of [transformers](#).



An illustration of the main components of the transformer model from the original paper, where layers were normalized after (instead of before) multiheaded attention

At the 2017 [NeurIPS](#) conference, [Google](#) researchers introduced the transformer architecture in their landmark paper "[Attention Is All You Need](#)".^[13] This paper's goal was to improve upon 2014 seq2seq technology, and was based mainly on the [attention](#) mechanism developed by Bahdanau et al. in 2014.^{[14][15]} The following year in 2018, [BERT](#) was introduced and quickly became "ubiquitous".^[16] Though the original transformer has both encoder and decoder blocks, BERT is an encoder-only model. Academic and research usage of BERT began to decline in 2023, following rapid improvements in the abilities of decoder-only models (such as GPT) to solve tasks via [prompting](#).^[17]

Although decoder-only [GPT-1](#) was introduced in 2018, it was [GPT-2](#) in 2019 that caught widespread attention because [OpenAI](#) claimed to have initially deemed it too powerful to release publicly, out of fear of malicious use.^[18] [GPT-3](#) in 2020 went a step further and as of 2025 is available only via [API](#) with no offering of downloading the model to execute locally. But it was the consumer-facing chatbot [ChatGPT](#) in late 2022 that received extensive media coverage and public attention by 2023.^[19] The 2023 [GPT-4](#) was praised for its increased accuracy and as a "holy grail" for its [multimodal](#) capabilities.^[20] OpenAI did not reveal the high-level architecture and the number of [parameters](#) of GPT-4. The release of ChatGPT led to an uptick in LLM usage across several research subfields of computer science, including robotics, software engineering, and societal impact work.^[17] In 2024, OpenAI released the [reasoning model OpenAI o1](#), which generates long chains of thought before returning a final answer.^[21] Many LLMs with parameter counts comparable to those of OpenAI's GPT series have been developed.^[22]

Since 2022, weights-available models have been gaining popularity, especially at first with [BLOOM](#) and [LLaMA](#), though both have restrictions on usage and deployment. [Mistral AI](#)'s open-weight models Mistral 7B and Mixtral 8x7B have a more permissive [Apache License](#). In January 2025, [DeepSeek](#) released DeepSeek R1, a 671-billion-parameter open-weight model that performs comparably to OpenAI o1 but at a much lower price per token for users.^[23]

Since 2023, many LLMs have been trained to be [multimodal](#), having the ability to also process or generate other types of data, such as images, audio, or 3D meshes.^[citation needed]

Open-weight LLMs have become more influential since 2023. Per Vake et al. (2025), community-driven contributions to open-weight models improve their efficiency and performance via collaborative platforms such as [Hugging Face](#).^[24]

Dataset preprocessing

See also: [List of datasets for machine-learning research § Internet](#)

Tokenization

As [machine learning](#) algorithms process numbers rather than text, the text must be converted to numbers. In the first step, a vocabulary is decided upon, then integer indices are arbitrarily but uniquely assigned to each vocabulary entry, and finally, an [embedding](#) is associated with the integer index. Algorithms include [byte-pair encoding](#) (BPE) and WordPiece. There are also special tokens serving as [control characters](#), such as [MASK] for masked-out token (as used in [BERT](#)), and [UNK] ("unknown") for characters not appearing in the vocabulary. Also, some special symbols are used to denote special text formatting. For example, "Ġ" denotes a preceding whitespace in [RoBERTa](#) and GPT and "##" denotes continuation of a preceding word in BERT.^[25]

For example, the BPE tokenizer used by the legacy version of [GPT-3](#) would split tokenizer: texts -> series of numerical "tokens" as

token izer : texts -> series of numerical " t o k e n s "

Tokenization also [compresses](#) the datasets. Because LLMs generally require input to be an [array](#) that is not [jagged](#), the shorter texts must be "padded" until they match the length of the longest one.

Byte-pair encoding

Main article: [Byte-pair encoding](#)

As an example, consider a tokenizer based on byte-pair encoding. In the first step, all unique characters (including blanks and [punctuation marks](#)) are treated as an initial set of [n-grams](#) (i.e.

initial set of uni-grams). Successively the most frequent pair of adjacent characters is merged into a bi-gram and all instances of the pair are replaced by it. All occurrences of adjacent pairs of (previously merged) n -grams that most frequently occur together are then again merged into even lengthier n -gram, until a vocabulary of prescribed size is obtained. After a tokenizer is trained, any text can be tokenized by it, as long as it does not contain characters not appearing in the initial-set of uni-grams.^[26]

Dataset cleaning

Main article: [Data cleansing](#)

In the context of training LLMs, datasets are typically cleaned by removing low-quality, duplicated, or toxic data.^[27] Cleaned datasets can increase training efficiency and lead to improved downstream performance.^[28] A trained LLM can be used to clean datasets for training a further LLM.^[29]

With the increasing proportion of LLM-generated content on the web, data cleaning in the future may include filtering out such content. LLM-generated content can pose a problem if the content is similar to human text (making filtering difficult) but of lower quality (degrading performance of models trained on it).^[1]

Synthetic data

Main article: [Synthetic data](#)

Training of largest language models might need more linguistic data than naturally available, or that the naturally occurring data is of insufficient quality. In these cases, synthetic data might be used.

Training

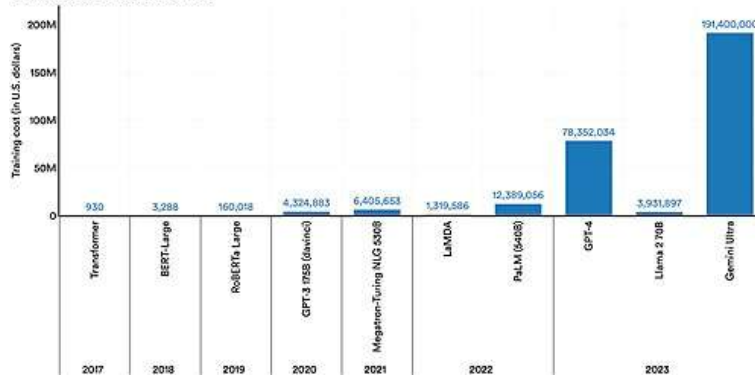
See also: [Fine-tuning \(machine learning\)](#)

An LLM is a type of [foundation model](#) (large X model) trained on language. LLMs can be trained in different ways. In particular, GPT models are first pretrained to predict the next word on a large amount of data, before being fine-tuned.^[4]

Cost

Estimated training cost of select AI models, 2017–23

Source: Epoch, 2023 | Chart: 2024 AI Index report



Substantial infrastructure is necessary for training the largest models. The tendency towards larger models is visible in the [list of large language models](#). For example, the training of GPT-2 (i.e. a 1.5-billion-parameter model) in 2019 cost \$50,000, while training of the [PaLM](#) (i.e. a 540-billion-parameter model) in 2022 cost \$8 million, and Megatron-Turing NLG 530B (in 2021) cost around \$11 million. The qualifier "large" in "large language model" is inherently vague, as there is no definitive threshold for the number of parameters required to qualify as "large".

Fine-tuning

Before being [fine-tuned](#), most LLMs are next-token predictors.^[30] The fine-tuning shapes the LLM's behavior via techniques like [reinforcement learning from human feedback](#) (RLHF) or [constitutional AI](#).^[31]

Instruction fine-tuning is a form of [supervised learning](#) used to teach LLMs to follow user instructions. In 2022, OpenAI demonstrated [InstructGPT](#), a version of GPT-3 similarly fine-tuned to follow instructions.^[32]

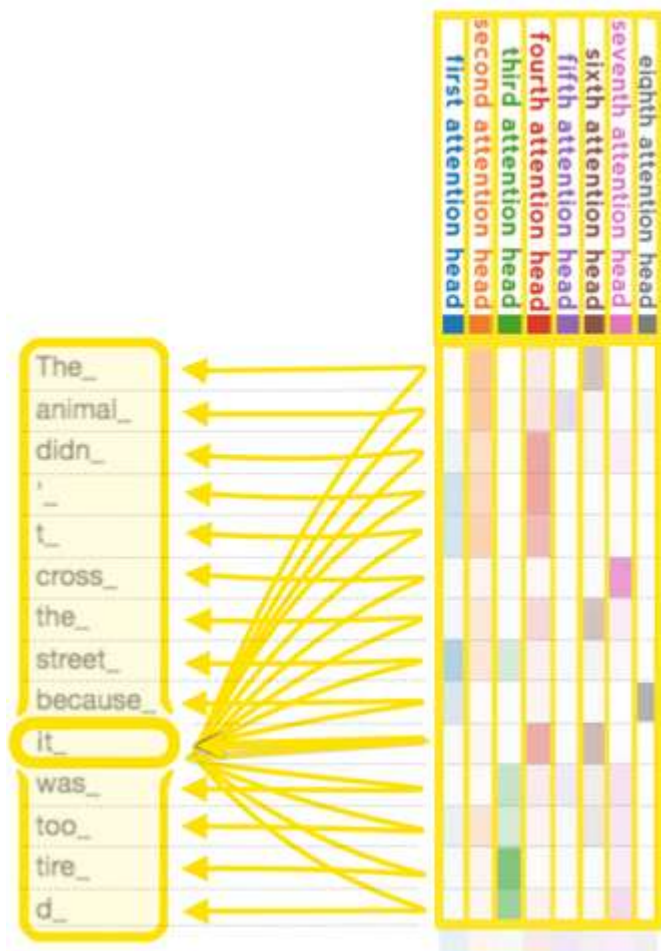
RLHF involves training a reward model to predict which text humans prefer. Then, the LLM can be fine-tuned through [reinforcement learning](#) to better satisfy this reward model. Since humans typically prefer truthful, helpful and harmless answers, RLHF favors such answers.^[33]

Architecture

LLMs are generally based on the [transformer](#) architecture, which leverages an [attention](#) mechanism that enables the model to process relationships between all elements in a sequence simultaneously, regardless of their distance from each other.^{[13][34]} Peng et al. (2023) proposed [state-space representation](#) models as an alternative.^[35]

Attention mechanism and context window

See also: [Attention \(machine learning\)](#)



When each head calculates, according to its own criteria, how much other tokens are relevant for the "it_" token, note that the second attention head, represented by the second column, is focusing most on the first two rows, i.e. the tokens "The" and "animal", while the third column is focusing most on the bottom two rows, i.e. on "tired", which has been tokenized into two tokens.^[36]

In order to find out which tokens are relevant to each other within the scope of the [context window](#), the attention mechanism calculates "soft" weights for each token, more precisely for its embedding, by using multiple attention heads, each with its own "relevance" for calculating its own soft weights. For example, the small (i.e. 117M parameter sized) [GPT-2](#) model has had twelve attention heads and a context window of only 1k tokens.^[37]

Autoregressive models, such as [GPTs](#), are trained to guess how a sequence continues; for example, whether the word sequence "I like to eat" is more likely to be followed by the word "bread" or the word "rocks". [Masked](#) models, such as BERT,^[38] are trained to guess parts that are missing from a sequence, such as whether the missing word in "I like to ___ roses" is more likely to be the word "smell" or the word "eat". The model's predictions are based on the properties of sequences within its training dataset.^[39]

Mixture of experts

Main article: [Mixture of experts](#)

A [mixture of experts](#) (MoE) is a [machine learning](#) architecture in which multiple specialized neural networks ("experts") work together, with a gating mechanism that routes each input to the most appropriate expert(s). Mixtures of experts can reduce inference costs, as only a fraction of the parameters are used for each input.^[40]

Parameter size

See also: [1.58-bit large language model](#)

Typically, LLMs are trained with single or half-precision [floating point numbers](#) (float32 and float16). One float16 has 16 bits, or 2 bytes, and so one billion parameters require 2 gigabytes. The largest models typically have more than 100 billion parameters, which places them outside the range of most consumer electronics.^[41]

Quantization

Post-training [quantization](#)^[42] aims to decrease the space requirement by lowering precision of the parameters of a trained model, while preserving most of its performance. Quantization can be further classified as *static quantization* if the quantization parameters are determined beforehand (typically during a calibration phase), and *dynamic quantization* if the quantization is applied during inference. The simplest form of quantization simply truncates all the parameters to a given number of bits: this is applicable to static as well as dynamic quantization, but loses much precision. Dynamic quantization allows for the use of a different quantization [codebook](#) per layer, either a lookup table of values or a linear mapping (scaling factor and bias), at the cost of foregoing the possible speed improvements from using lower-precision arithmetic.^[citation needed]

It is possible to fine-tune quantized models using [low-rank adaptation](#).^[43]

Extensibility

Beyond basic text generation, various techniques have been developed to extend LLM capabilities, including the use of external tools and data sources, improved reasoning on complex problems, and enhanced instruction-following or autonomy through prompting methods.

Prompt engineering

Main article: [Prompt engineering](#)

In 2020, [OpenAI](#) researchers demonstrated that their new model [GPT-3](#) could understand what format to use given a few rounds of Q and A (or other type of task) in the input data as example, thanks in part due to the RLHF technique. This technique, called *few-shot prompting*, allows LLMs to be adapted to any task without requiring fine-tuning.^[1] Also in 2022, it was found that the base GPT-3 model can generate an instruction based on user input. The generated instruction along with user input is then used as input to another instance of the model under a "Instruction: [...], Input: [...], Output:" format. The other instance is able to complete the output and often produces the correct answer in doing so. The ability to "self-instruct" makes LLMs able to [bootstrap](#) themselves toward a correct answer.^[44]

Dialogue processing (chatbot)

An LLM can be turned into a [chatbot](#) by specializing it for conversation. User input is prefixed with a marker such as "Q:" or "User:" and the LLM is asked to predict the output after a fixed "A:" or "Assistant:". This type of model became commercially available in 2022 with ChatGPT, a sibling model of InstructGPT fine-tuned to accept and produce dialog-formatted text based on GPT-3.5. It could similarly follow user instructions. Before the stream of User and Assistant lines, a chat context usually starts with a few lines of overarching instructions, from a role called "developer" or "system" to convey a higher authority than the user's input. This is called a "system prompt".^[citation needed]

Retrieval-augmented generation

[Retrieval-augmented generation](#) (RAG) is an approach that integrates LLMs with [document retrieval](#) systems. Given a query, a document retriever is called to retrieve the most relevant documents. This is usually done by encoding the query and the documents into vectors, then finding the documents with vectors (usually stored in a [vector database](#)) most similar to the vector of the query. The LLM then generates an output based on both the query and context included from the retrieved documents.^[45]

Tool use

Tool use is a mechanism that enables LLMs to interact with external systems, applications, or data sources. It can allow LLMs to, for example, fetch real-time information from an API or to execute code. A program separate from the LLM watches the output stream of the LLM for a special tool-calling syntax. When these special tokens appear, the program calls the tool accordingly and feeds its output back into the LLM's input stream.^[46]

Early tool-using LLMs were fine-tuned on the use of specific tools. But fine-tuning LLMs for the ability to read [API](#) documentation and call APIs correctly has greatly expanded the range of tools accessible to an LLM.^{[47][48]}

Agency

Main article: [AI agent](#)

An LLM is typically not an [autonomous agent](#) by itself, as it lacks the ability to interact with dynamic environments, recall past behaviors, and plan future actions. But it can be transformed into an agent by adding supporting elements: the role (profile) and the surrounding environment of an agent can be additional inputs to the LLM, while memory can be integrated as a tool or provided as additional input. Instructions and input patterns are used to make the LLM plan actions and tool use is used to potentially carry out these actions.^[49]

In the DEPS ("describe, explain, plan and select") method, an LLM is first connected to the visual world via image descriptions. It is then prompted to produce plans for complex tasks and behaviors based on its pretrained knowledge and the environmental feedback it receives.^[50]

The *Reflexion method* constructs an agent that learns over multiple episodes. At the end of each episode, the LLM is given the record of the episode, and prompted to think up "lessons learned", which would help it perform better at a subsequent episode. These "lessons learned" are stored as a form of long-term memory and given to the agent in the subsequent episodes.^[51]

[Monte Carlo tree search](#) can use an LLM as rollout heuristic. When a programmatic world model is not available, an LLM can also be prompted with a description of the environment to act as world model.^[52]

Multiple agents with memory can interact socially.^[53]

Chaining

Prompt chaining was introduced in 2022.^[54] In this method, a user manually breaks a complex problem down into several steps. In each step, the LLM receives as input a prompt telling it what to do and some results from preceding steps. The result from one step is then reused in a next step, until a final answer is reached. The ability of an LLM to follow instructions means that even non-experts can write a successful collection of stepwise prompts given a few rounds of trial and error.^{[55][56]}

A 2022 paper demonstrated a separate technique called [chain-of-thought prompting](#), which makes the LLM break the question down autonomously. An LLM is given some examples where the "assistant" verbally breaks down the thought process before arriving at an answer. The LLM mimics these examples and also tries to spend some time generating intermediate steps before providing the final answer. This additional step elicited by prompting improves the correctness of the LLM on relatively complex questions. On math word questions, a prompted model can exceed even fine-tuned GPT-3 with a verifier.^{[57][58]} Chain-of-thought can also be elicited by

simply adding an instruction like "Let's think step by step" to the prompt, in order to encourage the LLM to proceed methodically instead of trying to directly guess the answer.^[59]

Model-native reasoning

Main articles: [Reasoning model](#) and [Reflection \(artificial intelligence\)](#)

In late 2024, a new approach to LLM development emerged with "reasoning models".^[60] These are trained to generate step-by-step analysis before producing final answers, enabling better results on complex tasks, for instance in mathematics, coding and logic.^[61] OpenAI introduced this concept with their [o1](#) model in September 2024, followed by [o3](#) in April 2025. On the [International Mathematics Olympiad](#) qualifying exam problems, [GPT-4o](#) achieved 13% accuracy while o1 reached 83%.^[62]

In January 2025, the Chinese company [DeepSeek](#) released DeepSeek-R1, a 671-billion-parameter open-weight reasoning model that achieved comparable performance to OpenAI's o1 while being significantly more cost-effective to operate. Unlike proprietary models from OpenAI, DeepSeek-R1's open-weight nature allowed researchers to study and build upon the algorithm, though its training data remained private.^[63]

These reasoning models typically require more computational resources per query compared to traditional LLMs, as they perform more extensive processing to work through problems step by step.^[62]

Forms of input and output

Multimodality

See also: [Multimodal learning](#)

Multimodality means having multiple modalities, where a "[modality](#)" refers to a type of input or output, such as video, image, audio, text, [proprioception](#), etc.^[64] For example, [Google PaLM](#) model was fine-tuned into a multimodal model and applied to [robotic control](#).^[65] [LLaMA](#) models have also been turned multimodal using the tokenization method, to allow image inputs,^[66] and video inputs.^[67] [GPT-4o](#) can process and generate text, audio and images.^[68]

A common method to create multimodal models out of an LLM is to "tokenize" the output of a trained encoder. Concretely, one can construct an LLM that can understand images as follows: take a trained LLM, and take a trained image encoder E [multilayer perceptron](#) $f_{yf}(E(y))$ has the same dimensions as an encoded token. That is an "image token". Then, one can interleave text tokens and image tokens. The compound model is then fine-tuned on an image-text dataset. This basic construction can be applied with more sophistication to improve the model.

The image encoder may be [frozen](#) to improve stability.^[69] This type of method, where embeddings from multiple modalities are fused and the predictor is trained on the combined embeddings, is called *early fusion*.

Another method, called *intermediate fusion*, involves each modality being first processed independently to obtain modality-specific representations; then these intermediate representations are fused together.^[70] In general, cross-attention is used for integrating information from different modalities. As an example, the Flamingo model uses cross-attention layers to inject visual information into its pre-trained language model.^[71]

Non-natural languages

LLMs can handle [programming languages](#) similarly to how they handle natural languages. No special change in token handling is needed as code, like human language, is represented as plain text. LLMs can generate code based on problems or instructions written in [natural language](#). They can also describe code in natural language or translate it into other programming languages. They were originally used as a [code completion](#) tool, but advances have moved them towards [automatic programming](#). Services such as [GitHub Copilot](#) offer LLMs specifically trained, fine-tuned, or prompted for programming.^{[72][73]}

In [computational biology](#), transformer-based architectures, such as DNA LLMs, have also proven useful in analyzing biological sequences: [protein](#), [DNA](#), and [RNA](#). With proteins they appear able to capture a degree of "grammar" from the amino-acid sequence, by mapping that sequence into an [embedding](#). On tasks such as [structure prediction](#) and [mutational](#) outcome prediction, a small model using an embedding as input can approach or exceed much larger models using [multiple sequence alignments](#) (MSA) as input.^[74] ESMFold, [Meta Platforms'](#) embedding-based method for protein structure prediction, runs an order of magnitude faster than [AlphaFold2](#) thanks to the removal of an MSA requirement and a lower parameter count due to the use of embeddings.^[75] Meta hosts ESM Atlas, a database of 772 million structures of [metagenomic](#) proteins predicted using ESMFold.^[76] An LLM can also design proteins unlike any seen in nature.^[77] Nucleic acid models have proven useful in detecting [regulatory sequences](#),^[78] sequence classification, RNA-RNA interaction prediction, and RNA structure prediction.^[79]